

Unit I: Web Fundamentals (7 Hrs.)

1. Web Basics

History of Web(WWW)

Web Browsers and Web Servers

Website and Web Page (Static and Dynamic)

URL and URI

Internet

2. Overview of Web Protocols

HTTP

HTTPS

3. Domain Name and Hierarchy

DNS

Domain Registration

Web Hosting

4. Web Architecture

Client-Server Model

Multi-Tier Architecture (1-Tier, 2-Tier, 3-Tier)

5. Architecting Web Applications

Characteristics

Needs

Pros and Cons

6. Best Practices for Good Web Application Architecture

World Wide Web (WWW):

- The **World Wide Web (WWW)** is a system of interlinked **documents** and **multimedia** resources that are accessed via the **internet**.
- The Web allows us to browse websites, interact with content, and access a wide variety of information. The **Web** is one of the most powerful tools for communication, learning, and sharing knowledge in the modern world.
- The **World Wide Web** refers to all the **resources** and **users** on the internet that are using **HTTP (Hypertext Transfer Protocol)**. HTTP is the protocol that allows communication between web browsers and servers. It is the system through which we can access web pages and other resources on the internet.
- In simpler terms, the **World Wide Web** is a way to **exchange information** between computers connected to the internet. It connects millions of **webpages** and **documents**, forming a large collection of interactive resources like text, images, videos, and more. The Web allows users to click on links to move from one page to another, creating a **network** of information.

History of the World Wide Web (WWW)

1.Pre-Web Era (Before 1989)

- The **Internet** started as a network for military and research institutions (ARPANET in 1969).
- The **first email** was sent in 1971, enabling electronic communication over the network.
- **TCP/IP** protocol was developed in 1983, allowing different computers to communicate.
- The **Domain Name System (DNS)** was introduced in 1983 to convert website addresses into IP addresses.
- The web did not exist yet, and the Internet was used for data exchanges.
- The Internet was used primarily by scientists and the military for communication and research.

2. Web 1.0 – The "Library" Era (1989–2004)

- In 1989, **Tim Berners-Lee** proposed the **World Wide Web** to share documents online using links (hypertext).
- In 1990, the **first website** went live at **CERN**.
- In 1993, the **Mosaic** browser was released, allowing text and images to appear on the same page.
- Websites were **static** and primarily used for reading, with no interaction or content creation.
- The **dial-up internet** connection was slow and noisy.
- Websites like **Yahoo!** and **AOL** became popular as portals to the internet, serving as gateways for users.

3. Web 2.0 – The "Social" Era (2004–2016)

- The term "**Web 2.0**" became popular in 2004, marking a shift to a more interactive and social web.
- Platforms like **Facebook** (2004), **YouTube** (2005), and **Twitter** (2006) allowed users to create and share content.
- The launch of the **iPhone** in 2007 shifted the web to be **mobile-first**, allowing access from smartphones.
- **Cloud storage** services like **Google Drive** and **iCloud** allowed users to store files online.
- Websites became **interactive**, with features like comments, likes, and shares.
- **AJAX** technology improved website performance by making pages load faster and appear more like apps.

4.Web 3.0 – The "Smart" Era (2016–2025)

- Websites and platforms began to use **AI** and algorithms to **personalize** user experiences.
- The **Semantic Web** allowed computers to understand the meaning behind data, making searches more intuitive.
- **Blockchain** technology emerged, giving users more control over data and decentralizing power.
- AI-powered assistants like **Siri** and **Alexa** helped users navigate the web and control smart devices.
- **5G networks** enabled faster internet speeds, making websites more responsive.
- The **Internet of Things (IoT)** connected everyday devices like fridges, watches, and cars to the web.

5.Web 4.0– The "Agentic" Web (2026 and Beyond)

- **AI agents** will take actions for users, such as booking hotels or ordering food.
- **AI bots** will account for more than half of internet traffic, performing tasks like customer service.
- **Augmented Reality (AR)** and **Virtual Reality (VR)** will integrate the web into the physical world.
- Websites and apps will go beyond just providing information—they will **perform tasks** for users.
- **Blockchain** will continue to decentralize control, giving users more power over their data.
- The web will become more **immersive**, with **AR glasses** and **smart devices** creating interactive experiences.

The Impact of the World Wide Web

The **World Wide Web** has had a **great impact** on various aspects of life:

1. **Communication:** The Web has transformed communication, making it easier to connect with people worldwide through **email, social media, instant messaging,** and **video calls**.
2. **Business:** E-commerce has flourished, with businesses like **Amazon** and **Alibaba** reaching global audiences.
3. **Education:** Online learning platforms, educational resources, and e-books have made education more accessible.
4. **Entertainment:** The Web has become a major platform for entertainment, offering access to **streaming services** like **Netflix** and **YouTube**.

Challenges of the Web

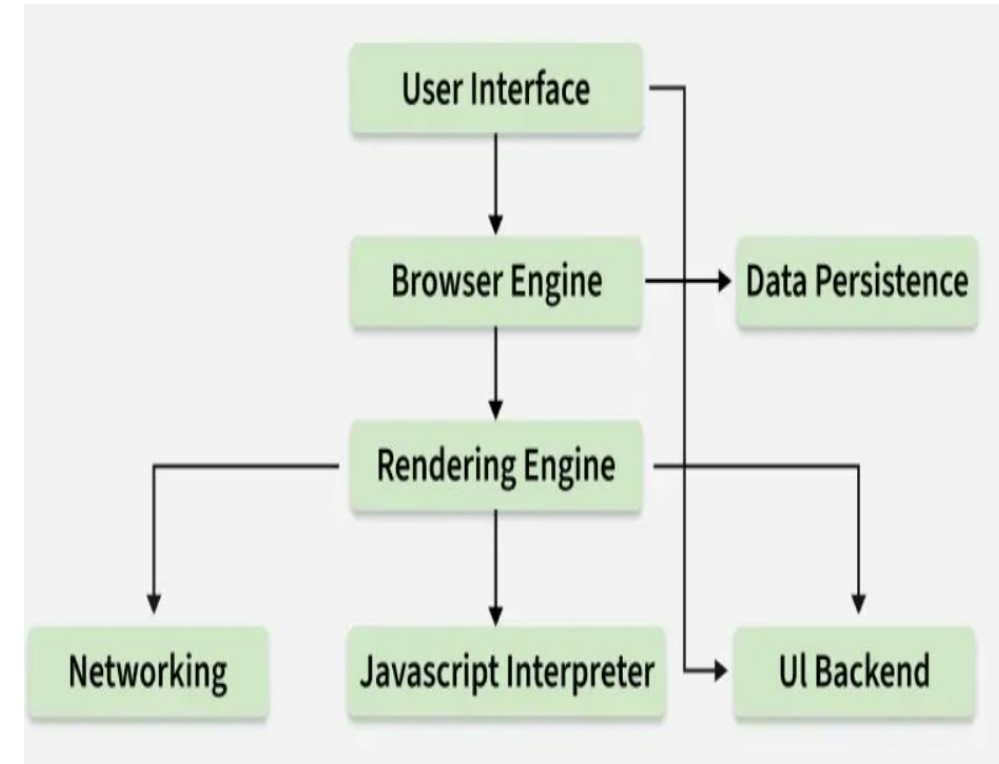
1. **Privacy:** Some websites collect **personal data** about you, such as your search history or location, and may share this information without asking for permission.
2. **Safety: Hackers** can try to steal your personal information or spread **viruses** through fake websites, links, or ads, putting your devices at risk.
3. **False Information:** Not everything you find on the web is true. It's important to **verify** the information and check if the website is **trustworthy** before believing or sharing it.
4. **Bullying: Cyberbullying** happens when people use the web to be mean or hurtful to others, which can negatively affect people's feelings and mental health.
5. **Too Much Screen Time:** Spending **too much time online** can affect your ability to focus on important tasks like **collegework**, and it can also impact your **sleep** and physical health.
6. **Access Issues:** Not everyone has **fast internet** access, especially in rural or less developed areas, making it difficult for some people to fully benefit from the web.

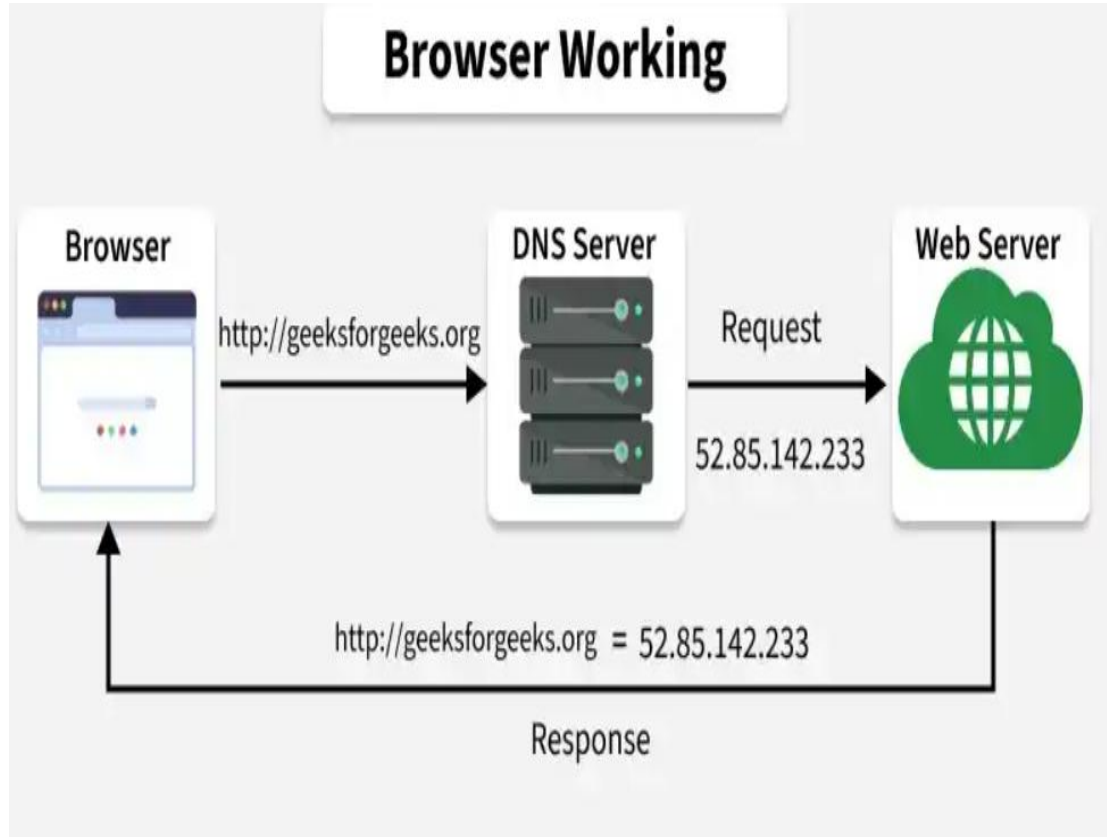
Web Browser:

- A **web browser** is a **software application** used to **retrieve, present, and navigate** information from the **World Wide Web (WWW)**.
- It operates in a **client-server architecture**, meaning it sends requests to web servers for data and displays the content (such as text, images, and interactive elements) in a format that is understandable and usable by humans.
- A web browser performs three primary functions:
 1. **Retrieval:** The browser uses a **URL (Uniform Resource Locator)** to locate a specific **web server** and requests content using the **HTTP** or **HTTPS** protocol. It essentially “asks” the server for a specific document, like a web page, by its address.
 2. **Rendering:** The browser’s **rendering engine** processes the code (e.g., HTML, CSS, JavaScript) sent by the server. It **arranges** and **formats** the content for display. For example, it converts raw HTML into readable text, formats it according to the style in CSS, and makes dynamic content work with JavaScript.
 3. **Navigation:** Browsers allow users to navigate between web pages using **hyperlinks** (links). This allows users to seamlessly transition from one document to another, creating the web's interconnected structure.

A web browser has several important components that help in processing and displaying web content:

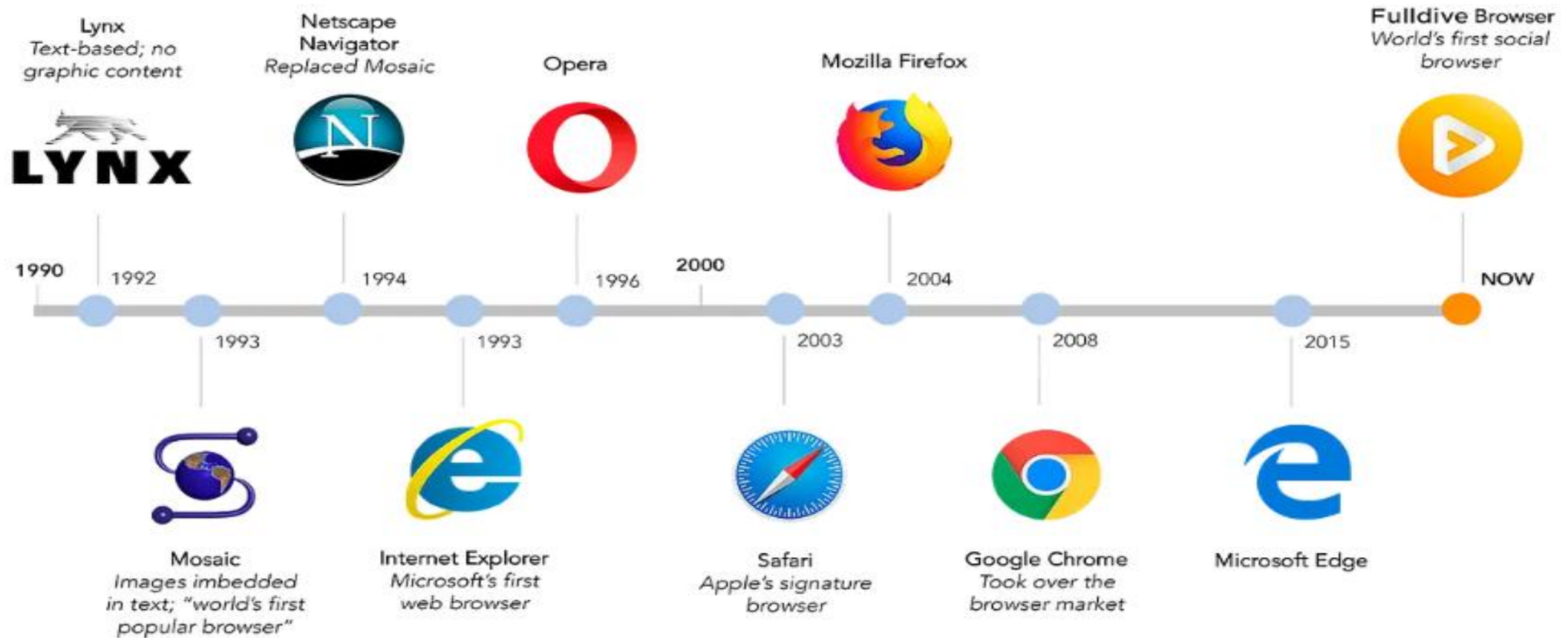
- 1.User Interface (UI):** This is what the user interacts with. It includes the **address bar**, **navigation buttons** (back, forward, reload), **tabs**, and **settings menus**.
- 2.Browser Engine:** This is the part of the browser that communicates with the rendering engine, managing actions like opening new tabs or loading new pages.
- 3.Rendering Engine:** The **rendering engine** interprets and displays the content (HTML, CSS, JavaScript) by translating it into a visual page. For example, Chrome uses **Blink**, while Firefox uses **Gecko**.
- 4.JavaScript Interpreter:** This component executes **JavaScript**, which allows the page to interact with users (e.g., opening pop-ups, validating forms).
- 5.Data Storage: Cookies, cache, and local storage** help the browser remember certain settings and data from websites. For example, cookies save login information, and cache stores images or scripts to speed up page loading.





- The user enters a website URL into the browser.
- The browser contacts a DNS (Domain Name System) server to convert the domain name into an IP address.
- Using the IP address, the browser sends a request to the web server .
- The server responds with web content such as HTML, CSS, images and scripts.
- The browser processes and renders this content into a readable webpage.

History of Web Browsers



1. **WorldwideWeb (1990):** **Tim Berners-Lee** created the first browser, **WorldWideWeb**, at **CERN**. It allowed users to view and edit web content, marking the birth of the **Web**.
2. **Mosaic (1993):** Developed by **NCSA(National Center for Supercomputing Applications.)**, **Mosaic** was the first graphical browser, introducing **images** alongside text, making the web more visually appealing.
3. **Netscape Navigator (1994):** **Marc Andreessen** created **Netscape**, the first popular browser. It featured **SSL encryption** and became widely used.
4. **Internet Explorer (1995):** **Microsoft** released **Internet Explorer (IE)**, which was bundled with **Windows** and led to the **browser wars** with Netscape.
5. **Opera (1996):** **Opera Software** introduced the **Opera browser**, known for its **speed** and **compact design**, as well as **mobile versions**.
6. **Mozilla Firefox (2002):** Based on **Netscape's open-source code**, **Firefox** focused on **privacy** and **security**, quickly becoming popular among users.
7. **Safari (2003):** **Apple** launched **Safari** for **Macintosh** systems, later available on **Windows**. It introduced **WebKit**, which influenced other browsers.
8. **Google Chrome (2008):** **Google** released **Chrome**, focused on **speed** and **simplicity**, and became the dominant browser with features like the **V8 JavaScript engine**.
9. **Microsoft Edge (2015):** **Microsoft** replaced **Internet Explorer** with **Edge**, later adopting the **Chromium engine** for better performance and compatibility.

web server:

- A **web server** is a system, either hardware or software, that stores and **delivers web content** to users over the **Internet**.
- It listens for incoming requests from **web browsers** (clients), processes them, and sends back the requested resources (such as **HTML, images, JavaScript** files) over **HTTP** or **HTTPS** protocols.

Key Functions :

- **Handling HTTP Requests:** Web servers receive **HTTP requests** from web browsers (clients). These requests may ask for web pages, images, or other resources.
- **Serving Web Content:** The server retrieves the requested content, such as HTML files or images, and sends them back to the client's browser for display.
- **Processing Dynamic Content:** Web servers often interact with server-side applications or databases to generate **dynamic content**. For example, displaying user-specific data, generating forms, etc.
- **Managing Security:** Web servers ensure secure communication using **SSL/TLS** for HTTPS, which encrypts data between the server and client to prevent unauthorized access.
- **Logging and Monitoring:** Web servers keep logs of incoming requests, errors, and other actions for analysis and troubleshooting.

Working

When a user enters a URL, the browser sends an HTTP request to the web server, which processes it and returns the required resources to display the page.



- **Client Request:** In the web browser (<https://www.facebook.com/>), the user enters a URL.
- **DNS Resolution:** The browser contacts a DNS server to obtain the IP address of the requested domain.
- **Establishing Connection:** Using the obtained IP address, the browser establishes a connection with the web server through TCP (and TLS in case of HTTPS).
- **Sending HTTP Request:** The browser sends an HTTP request (with headers and method like GET/POST) to the server.
- **Processing Request:** The web server receives the request, processes it, and may interact with backend services or databases.
- **Serving the Response:** The server sends back a response containing status codes and requested files (HTML, CSS, JavaScript, images).
- **Rendering the Web Page:** Based on the received data, the browser parses, executes scripts, and displays the web page to the user.

Server Type	Key Features	Ideal Use Case	Examples
Apache HTTP Server	Open-source, customizable, supports dynamic content (PHP, Python)	Flexible hosting for websites and applications	WordPress, Drupal, Joomla
Nginx	High-performance, reverse proxy, low resource usage, load balancing	High-traffic websites, static content serving	Pinterest, Netflix, GitHub
Microsoft IIS	Deep integration with Windows Server, supports ASP.NET	Enterprise-level applications on Windows	Corporate websites, enterprise solutions
LiteSpeed	High performance, security features, HTTP/2 and QUIC support	High-traffic sites requiring speed and security	Shared hosting providers
Caddy	Automatic HTTPS setup, simple configuration, built-in reverse proxy	Small projects, simple web apps with HTTPS	Personal websites, small businesses
Tomcat	Java-based, runs Java Servlets and JSP (Java Server Pages)	Java web applications, enterprise solutions	Java-based applications (e.g., Spring, Struts)
Cherokee	Lightweight, fast, easy configuration, supports HTTP/2	Minimalistic web applications, low-resource environments	Small websites, web apps
OpenLiteSpeed	Open-source, fast performance, built-in caching and security features	Fast and secure web hosting at low cost	Small to medium-sized business websites
Zeus	High-performance, security, load balancing for high-traffic sites	Large-scale, mission-critical websites	Corporate, high-traffic sites
Gunicorn	Python-based, WSGI-compatible, used with Python web frameworks	Serving Python web applications (Django, Flask)	Django, Flask apps

Web Page

- A **web page** is a single document displayed through a web browser, which is part of a website.
- It can include various elements such as **text**, **images**, **videos**, **links**, and **interactive elements** like buttons and forms.
- Web pages are primarily created using **HTML (HyperText Markup Language)** for structure, **CSS (Cascading Style Sheets)** for styling, and **JavaScript** for dynamic content and interactivity.

1.Static Web Pages

- A **static web page** is a web page that displays fixed content.
- The content does not change unless manually updated by the developer.
- Static web pages are often the simplest type of web pages, as they do not involve any server-side programming or dynamic elements.

Key Characteristics of Static Web Pages:

1. **Fixed Content:** The page shows the same content to all visitors.
2. **No Server-Side Processing:** Content is delivered as-is without interaction with a database.
3. **Simple and Fast:** Static pages are easy to create and load faster.
4. **Limited Interactivity:** User interaction is minimal, usually limited to navigation.

Advantages of Static Web Pages :

1. Fast loading times.
2. Simple to develop and maintain.
3. More secure with fewer potential vulnerabilities.

Disadvantages of Static Web Pages :

1. Lacks dynamic content or user personalization.
2. Harder to update, especially for large websites.

Use Cases: Personal portfolios, blogs, landing pages, and product info pages.

Dynamic Web Pages

- A **dynamic web page** is a web page that changes its content based on **user interaction**, **time**, or data from a **server** or **database**.
- Unlike static pages, dynamic pages generate content on-the-fly, offering a personalized and interactive experience for users.

Key Features:

1. **User Interaction:** Content updates based on user actions like clicking, scrolling, or filling out forms.
2. **Server-Side Processing:** Data is processed on the server (e.g., **PHP**, **ASP.NET**).
3. **Client-Side Processing:** Content can be updated using **JavaScript** without reloading the page.
4. **Database Integration:** Often pulls real-time data from a database to customize content (e.g., **e-commerce sites**, **social media**).

Advantages:

1. **Interactive:** Content changes based on user actions.
2. **Personalized:** Provides tailored experiences (e.g., product recommendations, user profiles).
3. **Real-Time Updates:** Content can be updated without needing to reload the page.

Disadvantages:

1. **Slower Load Times:** Due to server-side processing.
2. **Complex Development:** Requires more advanced programming and server resources.
3. **Security Risks:** Increased risk due to interactions with databases and user inputs.

Examples:

Amazon: Product recommendations and prices based on user preferences.

Facebook: Posts, likes, and notifications that change dynamically

Website:

- A **website** is a collection of **web pages** that are typically written in **HTML** (HyperText Markup Language).
- These web pages are linked together with **hyperlinks** and **hypertext** and share a common **design** and **interface**.
- Websites are hosted on a **web server**, a computer that is connected to the internet 24/7, making the website accessible globally.

Components of a Website:

1. **Web Host:** The physical location where the website's files are stored. A website can only exist if it is hosted on a **web server**.
2. **Address (URL):** A website's address is called its **URL** (Uniform Resource Locator). When a user enters the URL in a browser, the web server receives the request and sends the website's content to the browser.
3. **Homepage:** The **home page** is the first webpage a visitor sees. It sets the **look** and **feel** of the website and acts as a guide to the rest of the content on the site.

4. **Design:** The **design** refers to the overall appearance of the website, including elements such as **navigation menus**, **graphics**, **layout**, etc.
5. **Content:** The **content** of a website is the text, images, videos, and other media that make up the pages. High-quality content is key to making a website effective and attractive.
6. **Navigation Structure:** The **navigation structure** refers to how the pages are ordered and linked together. It is usually controlled by a **navigation menu** to help users move between pages.

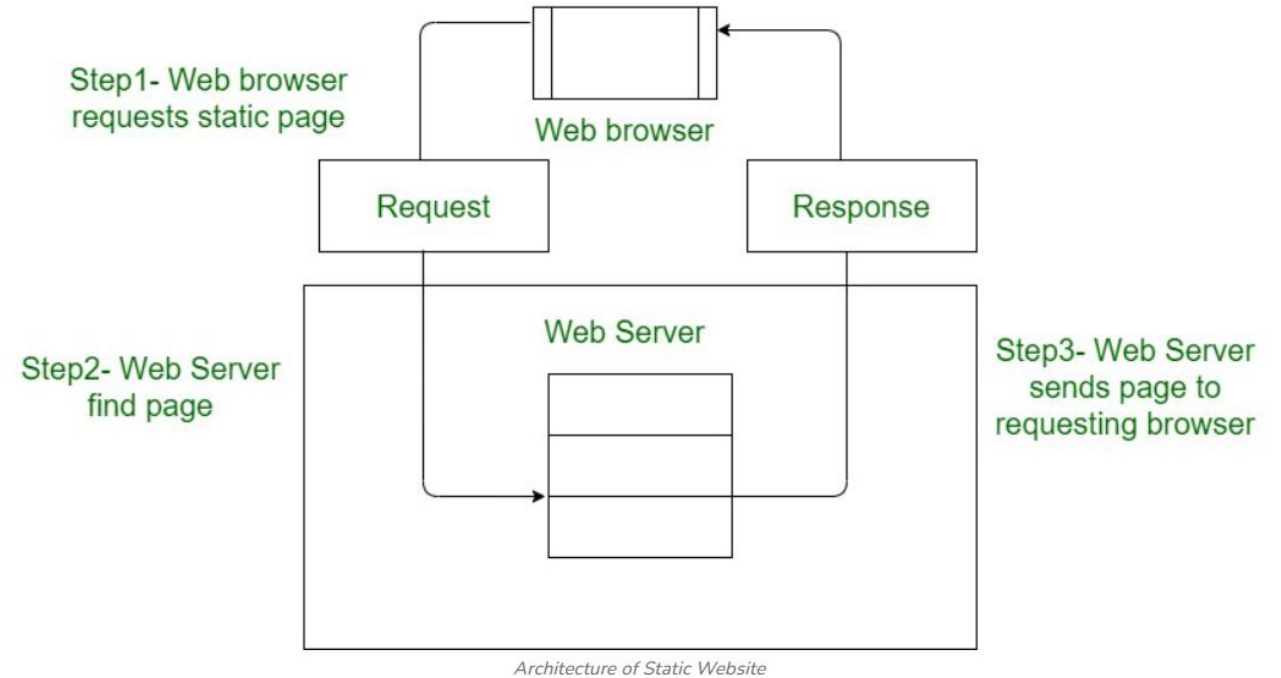
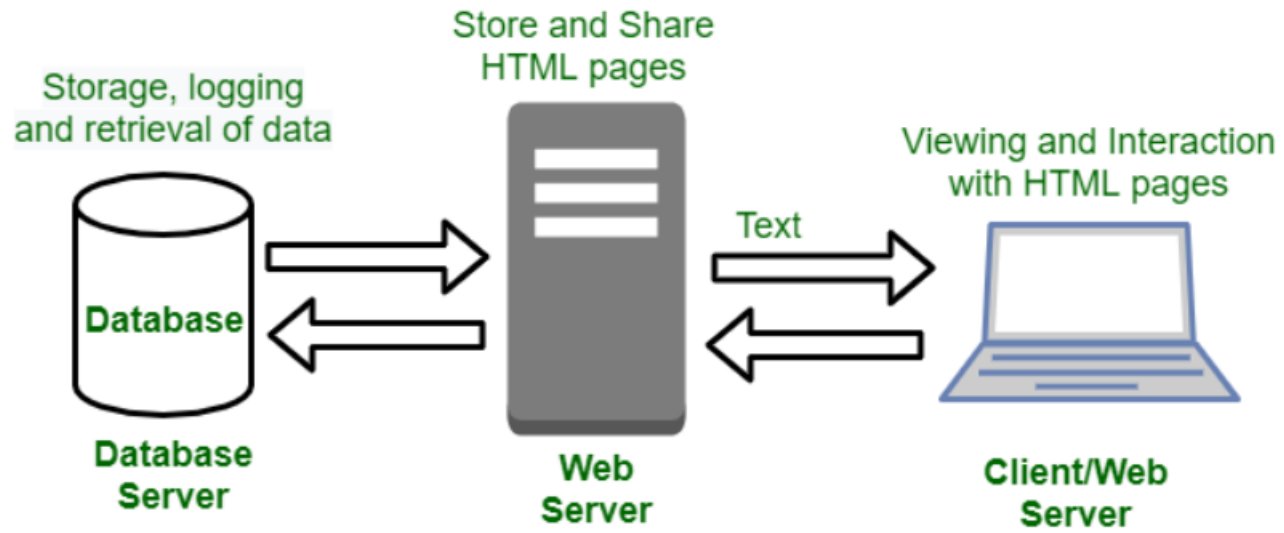
Common Categories of Websites:

1. **Blogs:** Run by individuals or small groups. Focus on topics like fashion, travel, or fitness. Blogs are a popular way to share personal opinions and can also be used to earn money.
2. **E-commerce Websites:** Websites that act as **online stores**, allowing users to buy and sell products or services. Includes **shopping carts**, **payment processing**, and product displays.
3. **Portfolio Websites:** Used by freelancers or professionals to showcase their work or skills. These sites serve as an online resume and work sample.

4. **Brochure Websites:** Simple websites used by small businesses. These are digital business cards displaying contact details and services.
5. **News and Magazine Websites:** Websites that provide up-to-date news, current events, or entertainment, such as news portals and online magazines.
6. **Social Media Websites:** Websites like **Facebook**, **Twitter**, or **Instagram** that allow users to create profiles, share content, and interact with others.
7. **Educational Websites:** Designed to deliver learning materials. These sites might offer online courses, educational resources, or interactive learning tools.
8. **Portal Websites:** Internal websites for organizations like schools or businesses. They often include **login systems** where users can access personalized information, emails, and other resources.

Difference Between Static and Dynamic Websites:

Static Website	Dynamic Website
Content of Web pages can not be change at runtime.	Content of Web pages can be changed.
No interaction with database possible.	Interaction with database is possible
It is faster to load as compared to dynamic website.	It is slower than static website.
Cheaper Development costs.	More Development costs.
No feature of Content Management.	Feature of Content Management System.
HTML, CSS, Javascript is used for developing the website.	Server side languages such as PHP, Node.js are used.
Same content is delivered everytime the page is loaded.	Content may change everytime the page is loaded.



comparison between a Website and a Webpage

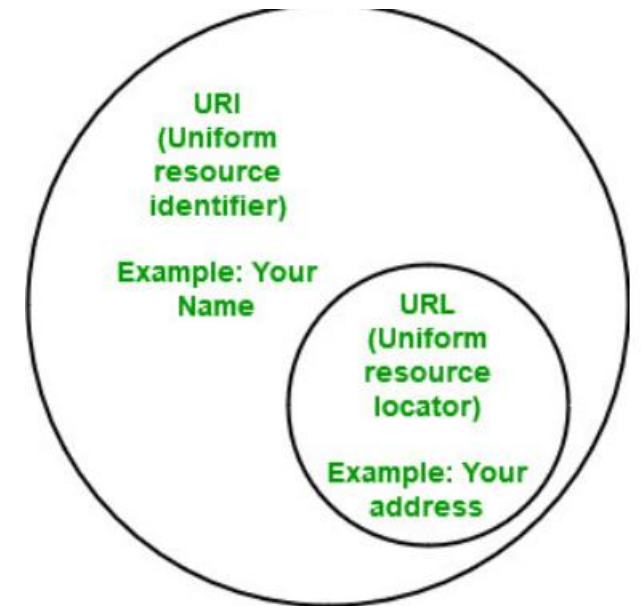
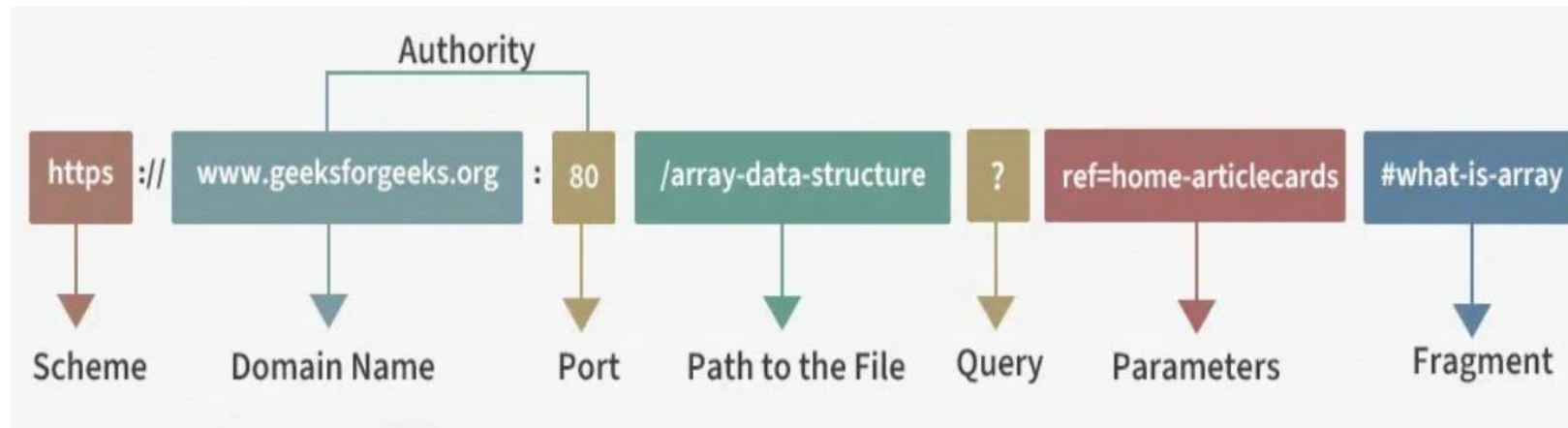
Website	Webpage
A collection of multiple web pages.	A single document displayed in the browser.
Requires more time and effort to develop.	Easier and quicker to develop once the website is structured.
Provides broad information across various pages.	Provides specific content related to a single topic or entity.
Used to establish online presence, credibility, and user experience.	Used to deliver focused information to users.
Includes content like text, images, videos, and hyperlinks across multiple pages.	Typically contains text, images, graphics, and links.
Typically accessed through a domain name (e.g., www.example.com).	Accessed through its own URL with an extension (e.g., www.example.com/page1.html).
Complex structure, involves programming for entire site.	Simple content structure and individual programming.
Designed to host various pages with linked content and resources.	Represents one page, often with links to other pages of the website.

What is a URI?

- A **URI** (Uniform Resource Identifier) is a string of characters that uniquely identifies a resource on the internet or within a network. It provides a way to reference a resource in a uniform manner, making it easier to access, reference, and interact with it.
- URI is a **broad term** that encompasses both **URLs** (Uniform Resource Locators) and **URNs** (Uniform Resource Names), which are used for identifying resources by location and by name, respectively.
- In simpler terms, a **URI** is a unique string of characters used to identify a resource, regardless of whether it's accessible via location, by name, or both.

What is a URL (Uniform Resource Locator)?

- A **URL** (Uniform Resource Locator) is a unique identifier used to locate a specific resource on the internet.
- It is commonly known as a **web address** and allows users to access websites and other resources by entering the URL into a browser's address bar or by clicking a link on a webpage.
- Structure of a URL



Protocol (or Scheme)

- Specifies the method used to access the resource.
- Common protocols include:
 1. **HTTP** (HyperText Transfer Protocol)
 2. **HTTPS** (Secure HTTP)
 3. **FTP** (File Transfer Protocol)
 4. **mailto** (for email links)
 5. **TELNET** (for remote computer access)

Domain Name (or Hostname):

- The address that identifies the server where the resource is located.
- **Example:** www.geeksforgeeks.org is the domain name.

Fragment (Optional):

- Refers to a specific section of a page.
- Starts with a # symbol
- Example: #what-is-array.

Port (Optional):

- Specifies the port number used for the connection.
- This is often not visible in the URL but is necessary for non-default ports.
- **Example:** :80 for HTTP, :443 for HTTPS.

Path:

- It Refers to the specific location of the file or page on the server.
- **Example:** /dsa/array-data-structure-guide/

Query (Optional)

- Used in dynamic pages to pass parameters to the server.
- Starts with a ? followed by key-value pairs separated by &.m
- **Example:** ?ref=home-articlecard.

Real-life Example of a URL:

URL: <https://www.amazon.com/products/item1?color=blue&id=123#reviews>

Scheme/Protocol: https://

Subdomain: www

Domain Name: amazon.com

Path: /products/item1

Query String: ?color=blue&id=123

Fragment: #reviews

URN (Uniform Resource Name)

- A **URN (Uniform Resource Name)** is a type of **URI (Uniform Resource Identifier)** that uniquely identifies a resource without providing its **location** or **method of access**.
- Unlike a **URL** (Uniform Resource Locator), which specifies where and how to find a resource, a **URN** is purely a **name** that can be used to identify a resource, regardless of its location or how it is accessed.
- A typical URN has the following format:

urn:<namespace identifier>:<resource identifier>

Namespace Identifier: Defines the domain of the resource (e.g., ISBN for books, DOI for academic papers).

Resource Identifier: Uniquely identifies the resource within that namespace (e.g., a specific book's ISBN or a paper's DOI).

Example of URN:

urn:isbn:0451450523

//This URN refers to a specific book identified by its ISBN number, without specifying where it is located or how to access it.

urn:uuid:123e4567-e89b-12d3-a456-426614174000

//This URN represents a Universal Unique Identifier (UUID) for a specific resource.

Difference Between URL and URI

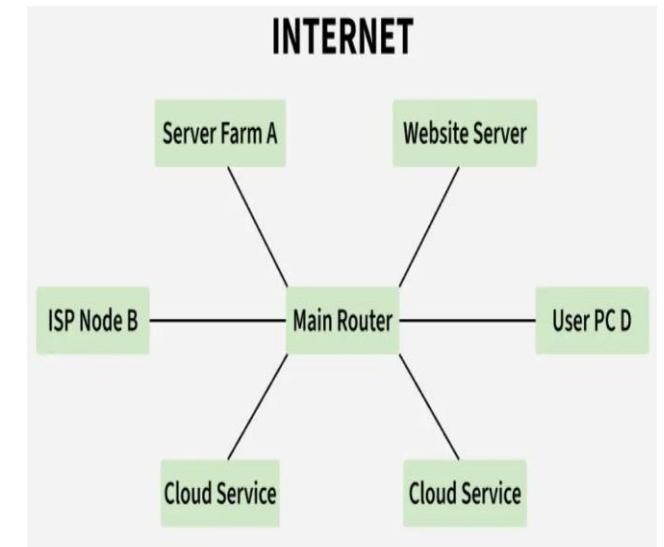
URL (Uniform Resource Locator)	URI (Uniform Resource Identifier)
A specific type of URI that provides the location and access method for a resource.	A general identifier for a resource, which can specify location, name, or both.
Specifies where and how to access a resource (location + protocol).	Identifies a resource, but not necessarily how to access it.
Includes protocol (HTTP, HTTPS), domain, path, query, and sometimes fragment.	Can include scheme, authority, path, query, and fragment, but does not necessarily have access method.
Yes, specifies the location of the resource.	Not necessarily, can be just a name or identifier without specifying location.
Required (e.g., HTTP, FTP).	Not required; can exist without specifying a protocol.
A subset of URI.	URI is the broader concept that includes URLs and other identifiers.
Example: <code>https://www.example.com/index.html</code>	Example: <code>urn:isbn:0451450523</code>
Primarily used to locate and access resources on the web.	Used to identify resources by name, location, or both.
Specifies the access method (e.g., HTTP, FTP).	Does not necessarily specify an access method.

Internet:

- The Internet is a global system of interconnected computer networks that communicate using standard protocols, primarily the Internet Protocol (IP).
- It enables the sharing of data, resources, and services, connecting billions of devices and users around the world.

Features :

1. **Global Reach:** Connects users worldwide.
2. **Accessibility:** Available to anyone with a device and connection.
3. **Communication:** Enables email, video calls, and social media.
4. **Information Sharing:** Access to vast resources and websites.
5. **E-commerce:** Facilitates online buying and selling.
6. **Research:** A valuable tool for academic and professional studies.
7. **Social Networking:** Platforms like Facebook and Instagram.
8. **Collaboration:** Tools for teamwork and file sharing.
9. **Entertainment:** Streaming of videos, music, and games.
10. **Security:** Protected by encryption, firewalls, and antivirus software.

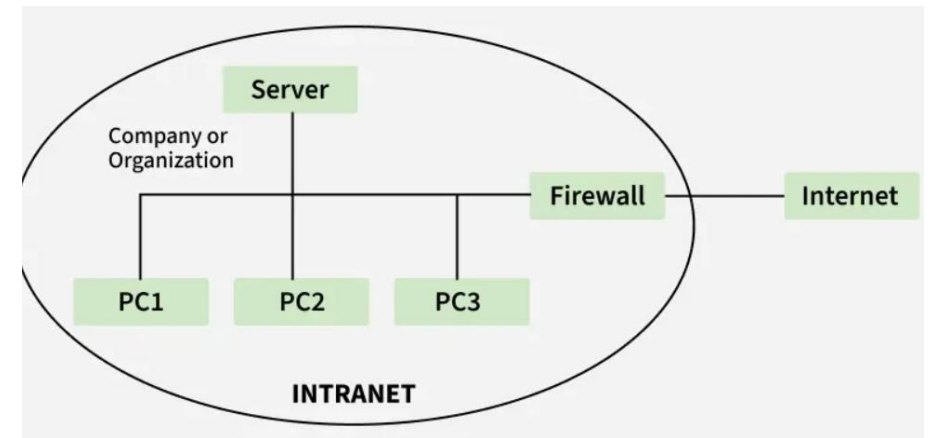


Intranet:

- An **Intranet** is a private network within an organization that uses internet protocols to securely share data, communication, and resources among employees or members.
- Unlike the public internet, access to an intranet is restricted to authorized users.

Key Features of Intranet:

1. **Private Access:** Restricted to authorized users within the organization.
2. **Communication:** Internal messaging, email, and forums.
3. **File Sharing:** Secure sharing and storage of documents.
4. **Collaboration Tools:** Shared calendars, project management tools.
5. **Security:** Protected by firewalls and encryption.
6. **Customization:** Tailored to organizational needs.
7. **Remote Access:** Available via VPN or secure portals.

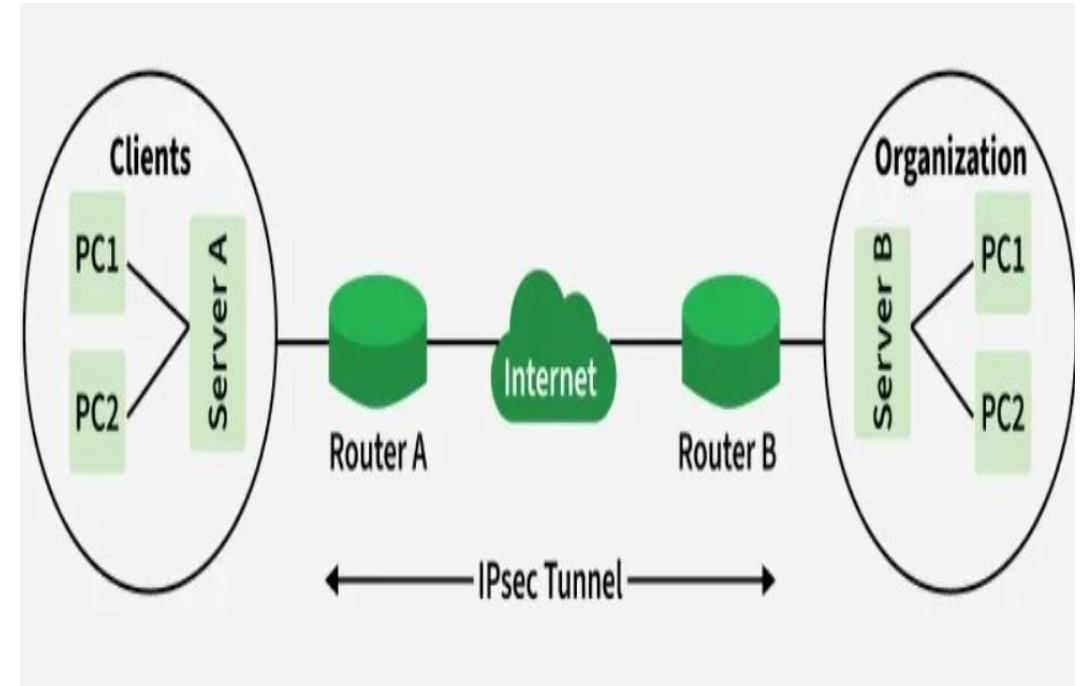


Extranet:

- An **Extranet** is a private network that allows external users (such as business partners, suppliers, or customers) to access certain parts of an organization's internal network.
- It extends an intranet to authorized external parties while maintaining security.

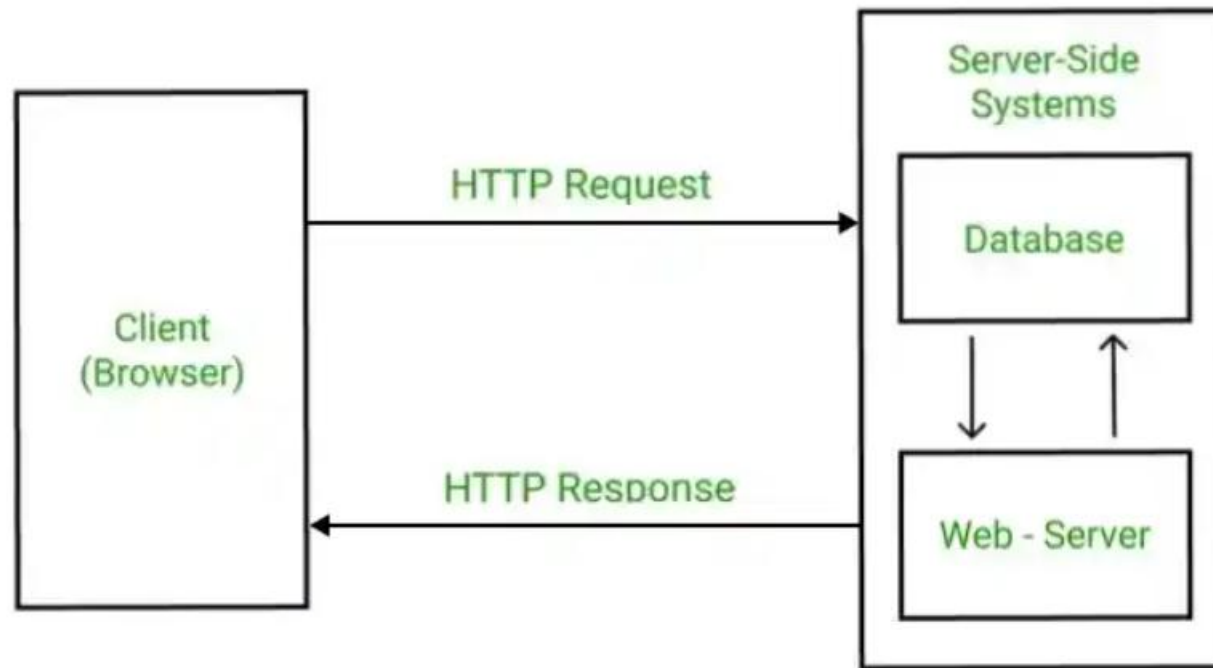
Key Features of Extranet:

1. **Controlled Access:** External users are granted access to specific resources or information.
2. **Collaboration:** Facilitates collaboration between the organization and external partners.
3. **Secure Communication:** Uses encryption and secure protocols to protect data shared externally.
4. **File Sharing:** Allows sharing of documents and files between the organization and external parties.
5. **Customizable Permissions:** Access can be restricted based on user roles, ensuring data privacy.
6. **Remote Access:** External users can access the network securely from any location.
7. **Efficiency:** Improves workflow and coordination between the organization and external collaborators.



HTTP - Hypertext Transfer Protocol

- HTTP stands for **Hypertext Transfer Protocol**.
- It is the protocol that allows communication between **web browsers** (like Google Chrome or Firefox) and **web servers**.
- It is a set of rules or standards that dictate how data is requested and transferred between clients (browsers) and servers to ensure that websites load correctly and display content.



Features of HTTP

- 1.Stateless:** Each request is independent; no data is retained from previous interactions.
- 2.Request-Response Model:** Clients (browsers) send requests, and servers respond with data.
- 3.Client-Server Architecture:** The client makes requests, and the server serves resources.
- 4.Human-Readable:** HTTP headers and data are text-based and easy to debug.
- 5.Supports Multiple Methods:** Includes methods like GET, POST, PUT, DELETE for different actions.
- 6.Use of Headers:** Sends metadata (like content type, length) in headers.
- 7.Supports Cookies:** Stores session data for users (e.g., login info).
- 8.Text-Based:** Messages are sent as plain text for easy reading.
- 9.Status Codes:** Response includes codes like 200 OK, 404 Not Found to indicate request status.
- 11.Universality:** Supported across all web browsers and servers.

How HTTP works

1.Open Web Browser: The user launches a web browser and types in a URL (e.g., www.facebook.com).

2.DNS Lookup: The browser sends a query to a DNS (Domain Name System) server to resolve the domain name into an IP address, similar to looking up the website's "phone number."

3.Send HTTP Request: Once the IP address is obtained, the browser sends an HTTP request to the server at that address, asking for the resources needed to display the page (like text, images, and videos).

4.Server Response: The web server processes the HTTP request and returns an HTTP response, which includes the requested resources (such as HTML, CSS, JavaScript, and media files) that the browser will use to render the page.

5.Rendering the Web Page: The browser receives the resources and uses them to render the webpage and display it on the user's screen.

6.Connection Close: After the page is fully loaded, the connection between the browser and the server is closed. If the user requests another page, a new connection will be established.

Advantages of HTTP

- 1.Low Resource Usage:** It uses less memory and CPU because it only opens a connection when needed.
- 2.Less Traffic:** With fewer connections, the network is less congested.
- 3.Faster Connections:** After the first connection, future requests are quicker since no extra setup is required.
- 4.Easy Error Reporting:** Errors can be reported without closing the connection, making it more efficient.
- 5.Multiple Requests at Once:** HTTP can send and receive several requests at the same time, improving speed.

Disadvantages of HTTP

1.Uses More Power: It's not efficient for devices on weak or mobile networks, as it consumes more power.

2.Security Risks: HTTP doesn't encrypt data, so information can be intercepted by others.

3.Inefficient on Mobile: It drains battery and slows down on mobile devices with cellular networks.

4.Not Secure for Sensitive Data: Personal data, like passwords, can be intercepted during transfer.

5.Slower Server Performance: The server has to wait to send all data before handling other requests, which can slow things down.

HTTPS:

- HTTPS stands for HyperText Transfer Protocol Secure.
- It is the most common protocol for sending data between a web browser and a website.
- HTTPS is the secure variant of HTTP and is used to communicate between the user's browser and the website, ensuring that data transfer is encrypted for added security.

Key Features:

- **Encryption:** It encrypts data between the browser and server to protect it from being intercepted.
- **Authentication:** Verifies the identity of the website using SSL/TLS certificates, ensuring you're communicating with the legitimate site.
- **Data Integrity:** Ensures that the data hasn't been altered during transmission.
- **Secure Communication:** Protects sensitive data like passwords and credit card info.
- **Padlock Icon:** A browser padlock indicates a secure connection.
- **SEO Benefit:** Websites with HTTPS are ranked higher by search engines.

How HTTPS Works:

Browser Requests a Secure Connection:

- When you visit a website using **HTTPS**, your browser first sends a request to the server (e.g., <https://facebook.com>).
- The browser asks the server for its **SSL/TLS certificate** to establish a secure connection.

SSL/TLS Handshake:

- The server responds by sending its **SSL/TLS certificate**, which contains:
 - **Public Key**: Used to encrypt data.
 - **Certificate Authority (CA)**: Verifies that the server is legitimate.
- The browser checks the certificate for validity and authenticity (whether it's issued by a trusted authority and whether it has expired).
- If valid, the browser and server use the certificate to establish a **secure connection**.

Session Key Creation:

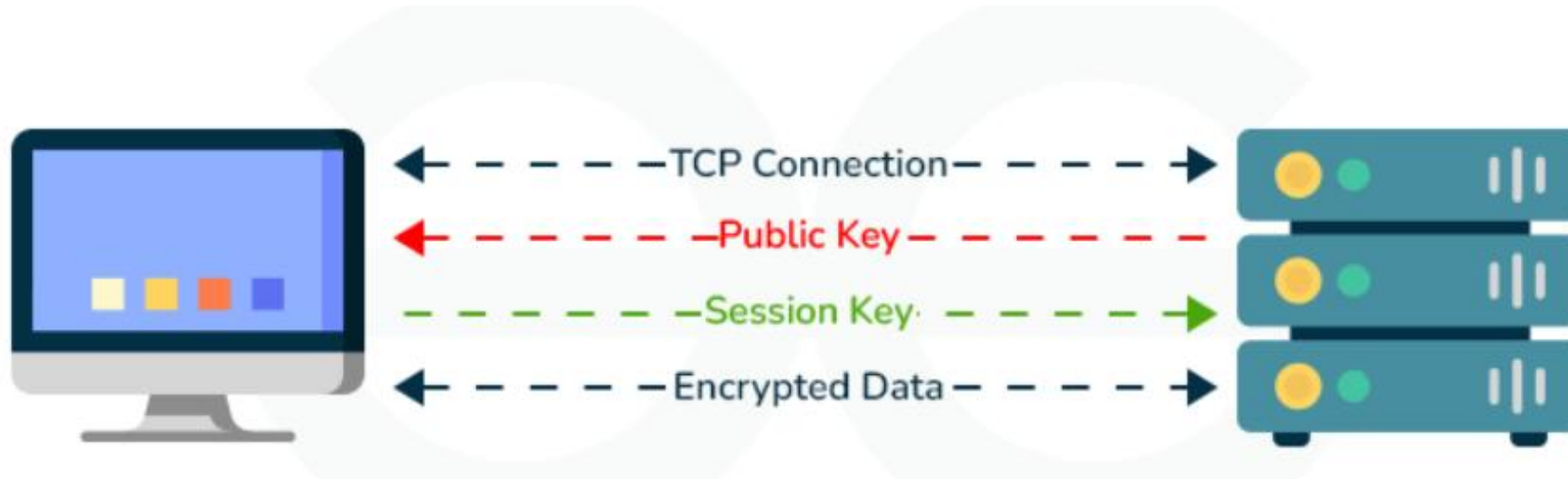
- The browser and server **agree on a "session key"** using a process called **asymmetric encryption**.
 - The server sends a random number encrypted with the **public key**.
 - The browser decrypts it with the corresponding **private key**.
- Both sides now have a shared **session key** used to encrypt all further communication.

Encrypted Data Transfer:

- From this point onward, all data exchanged between the browser and the server is **encrypted** using the session key.
- This ensures that even if someone intercepts the data, they can't read or alter it without the key.

Data Integrity: HTTPS also ensures **data integrity** by using **checksums**. If any data is altered during transmission, the connection is dropped and an error is shown.

Secure Closure: After the communication is complete, the session is securely closed.



Advantages of HTTPS over HTTP:

1. **Encryption:** HTTPS encrypts data, protecting it from interception, while HTTP sends data in plaintext.
2. **Authentication:** HTTPS verifies the server's identity, preventing fake sites, while HTTP offers no verification.
3. **Data Integrity:** HTTPS ensures data isn't tampered with during transfer, unlike HTTP.
4. **User Trust:** HTTPS shows a padlock icon, increasing trust, whereas HTTP doesn't signal security.
5. **SEO Benefit:** HTTPS is favored by search engines, improving rankings.
6. **Protection from Attacks:** HTTPS guards against **man-in-the-middle** attacks, unlike HTTP.
7. **Privacy:** HTTPS keeps browsing data private; HTTP exposes it to third parties.
8. **Modern Web Features:** HTTPS is required for advanced features like **geolocation** and **push notifications**.

comparison between HTTP and HTTPS:

Feature	HTTP	HTTPS
Security	No encryption, data is open	Data is encrypted and secure
Authentication	Doesn't verify the website	Verifies the website's identity
Data Integrity	Data can be changed during transfer	Ensures data is not changed
Port	Uses port 80	Uses port 443
URL Prefix	http://	https://
SEO Ranking	Lower ranking	Higher ranking in search results
Browser Indication	No security symbol	Shows a padlock symbol for security
Privacy	Data can be seen by others	Keeps browsing private
Vulnerability to Attacks	Easy to hack	Harder to hack
Required for Modern Features	Doesn't support some features	Needed for features like push notifications
Trust	Users may not trust it	Users trust it more

Domain Name

- A **domain name** is a human-readable address used to identify and access websites on the internet.
- Every website has a unique **IP address** (e.g., **142.250.190.46**), which is a numeric identifier.
- However, remembering long and complex numbers is not practical for users, which is why **domain names** are used to make web addresses easier to access.
- In simpler terms: **A domain name** is the name of a website.
- **Examples** : google.com, facebook.com, bbc.co.uk
- **Real life analogy** : Think of the internet like a **city**:
 - Every **house** (website) has a **house number** → **IP Address** (e.g., 192.168.1.1).
 - Every **house** also has a **name/label** → **Domain Name** (e.g., google.com).
 - The **phonebook** that converts names to numbers → **DNS (Domain Name System)**.
- Just like a city's phonebook matches a name to a phone number, the **DNS** translates the domain name into the corresponding IP address that points to the correct website.

Purpose of Domain Names

- The primary purpose of domain names is to **replace IP addresses** with easily readable names. This makes it simpler for people to access websites without needing to memorize long strings of numbers.
- Without domain names, the internet would be like a telephone directory where you need to dial numbers (IP addresses) to reach a particular destination. Instead, domain names serve as **user-friendly addresses** to make browsing the internet easier.

Without Domain Name

142.250.190.46

31.13.79.35

151.101.1.140

With Domain Name

google.com

facebook.com

reddit.com

- **Simplifies Navigation:** Replaces complex IP addresses with human-readable names, making it easier for users to access websites.
- **Establishes Online Identity:** Serves as the unique **identifier** for a website, creating a distinct presence on the internet.
- **Supports Branding:** Enhances **brand recognition** and credibility, helping businesses or individuals stand out online.
- **Improves SEO:** A relevant domain name with keywords can help boost **search engine rankings** and visibility.
- **Enhances Trust:** A custom domain name provides a **professional appearance**, which increases the **trustworthiness** of a website.

Who Manages Domain Names?

The management of domain names is handled by several organizations at different levels:

Organization	Role
ICANN	The Internet Corporation for Assigned Names and Numbers oversees the global domain name system and ensures that domain names are unique and properly registered.
Registries	These organizations manage specific Top-Level Domains (TLDs). For example, Verisign manages the .com TLD.
Registrars	Companies (like GoDaddy, Namecheap, Google Domains) that sell domain names to individuals and businesses. They act as intermediaries between registrants and registries.
Registrant	The person or organization who buys and owns the domain name. They have the right to use and renew the domain.

Characteristics of a Domain Name

- Must be **unique** to ensure no two websites share the same domain.
- Maximum length is **253 characters**, including the TLD.
- Can contain **letters, numbers, and hyphens** (no spaces or special characters).
- Domain names are **not case-sensitive** (e.g., Google.com = google.com).
- **Cannot** start or end with a hyphen (-).

What is DNS?

- The **Domain Name System (DNS)** is a system that translates **human-readable domain names** (like **www.google.com**) into **machine-readable IP addresses** (like **192.0.2.1**).
- It acts as the **phonebook of the internet**, allowing users to access websites and other online resources using easily memorable domain names instead of complex numeric IP addresses.
- Every device connected to the internet has a unique IP address that allows it to communicate with other devices. However, remembering a string of numbers for every website would be difficult, which is why DNS exists to simplify this process.

Domain Name and their types

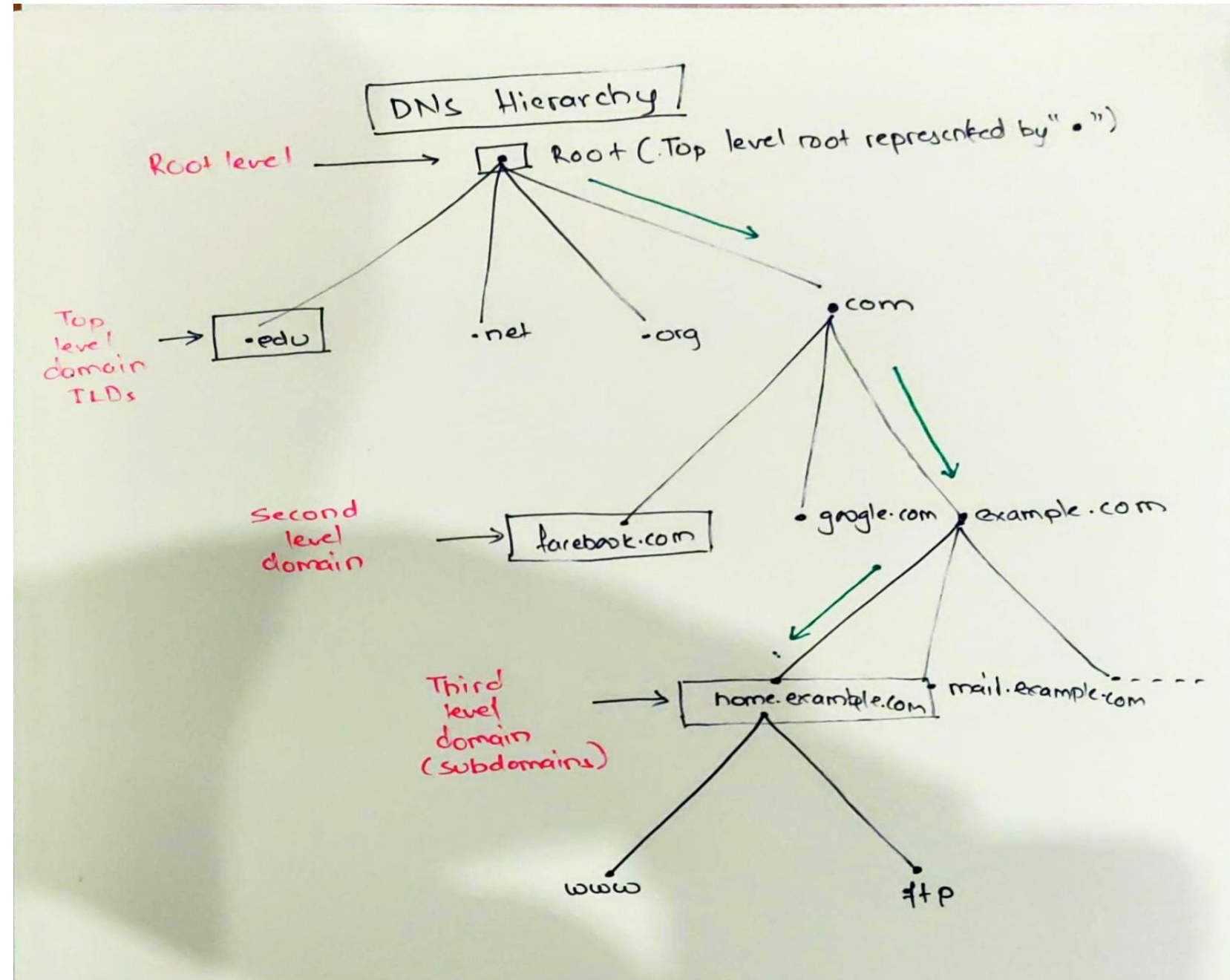
Domain Name	Type	Belongs To
nepal.gov.np	Government	Government of Nepal
moe.gov.np	Government	Ministry of Education, Nepal
moha.gov.np	Government	Ministry of Home Affairs, Nepal
tu.edu.np	Educational	Tribhuvan University
kathmandu.edu.np	Educational	Kathmandu University
himalaya.com.np	Commercial	Himalaya Internet Service
websurfer.com.np	Commercial	Web Surfer Pvt. Ltd.
himalayan.org.np	Non-profit	Himalayan Trust Nepal
google.com	Commercial	Google LLC
wikipedia.org	Non-profit	Wikimedia Foundation
nasa.gov	Government	NASA, USA
harvard.edu	Educational	Harvard University
bbc.co.uk	Country-specific (UK)	BBC, UK

Domain Name Hierarchy

- The **Domain Name System (DNS)** is structured in a hierarchical format, where domain names are broken down into multiple levels.
- Each level represents a different **part** of the domain name, starting from the **most general** to the **most specific**.
- This hierarchy helps to organize and manage the global domain name system efficiently.

1. Root Domain:

This is the highest level in the DNS hierarchy. It is typically represented as a dot (".") and is managed by **ICANN** (Internet Corporation for Assigned Names and Numbers).



2. Top-Level Domains (TLDs)

- TLDs represent the highest level in the domain name hierarchy. They appear as the last part of a domain name, following the dot (e.g., **.com**, **.org**).

➤ Subtypes of TLDs:

1. **Generic Top-Level Domains (gTLDs)**: These are not tied to any specific country and can be used globally. They are the most commonly used domain types.

Examples:

1. **.com** - Commercial businesses.
2. **.org** - Non-profit organizations.
3. **.net** - Network infrastructure and service providers.
4. **.info** - Information-based websites.

2. **Country-Code Top-Level Domains (ccTLDs)**: These are specific to particular countries or regions, represented by two letters.

Examples:

1. **.np** - Nepal.

2. **.in** - India.

3. **.uk** - United Kingdom.

4. **.us** - United States.

3. Sponsored Top-Level Domains (sTLDs): These are managed by specific organizations, and their use is often restricted to particular industries or communities.

Examples:

1. **.edu** - Educational institutions.
2. **.gov** - Government entities.
3. **.mil** - Military organizations.

4. New gTLDs: Introduced recently to provide more specific choices for industries, activities, or interests.

Examples:

1. **.tech** - Technology-related websites.
2. **.shop** - E-commerce websites.
3. **.blog** - Blogs and personal websites.
4. **.app** - Application-related websites.

3.Second-Level Domains (SLDs)

- The **Second-Level Domain** is the part of the domain name immediately to the left of the TLD.
- It is usually the name chosen by the individual or organization registering the domain.

Characteristics:

- **Unique:** Each SLD must be unique across the internet.
- **Allowed Characters:** Letters, numbers, and hyphens (but not at the start or end).
- **Memorability:** SLDs should be short, memorable, and easy to recall.

Examples:

- In **google.com**, "google" is the SLD.
- In **facebook.com**, "facebook" is the SLD.

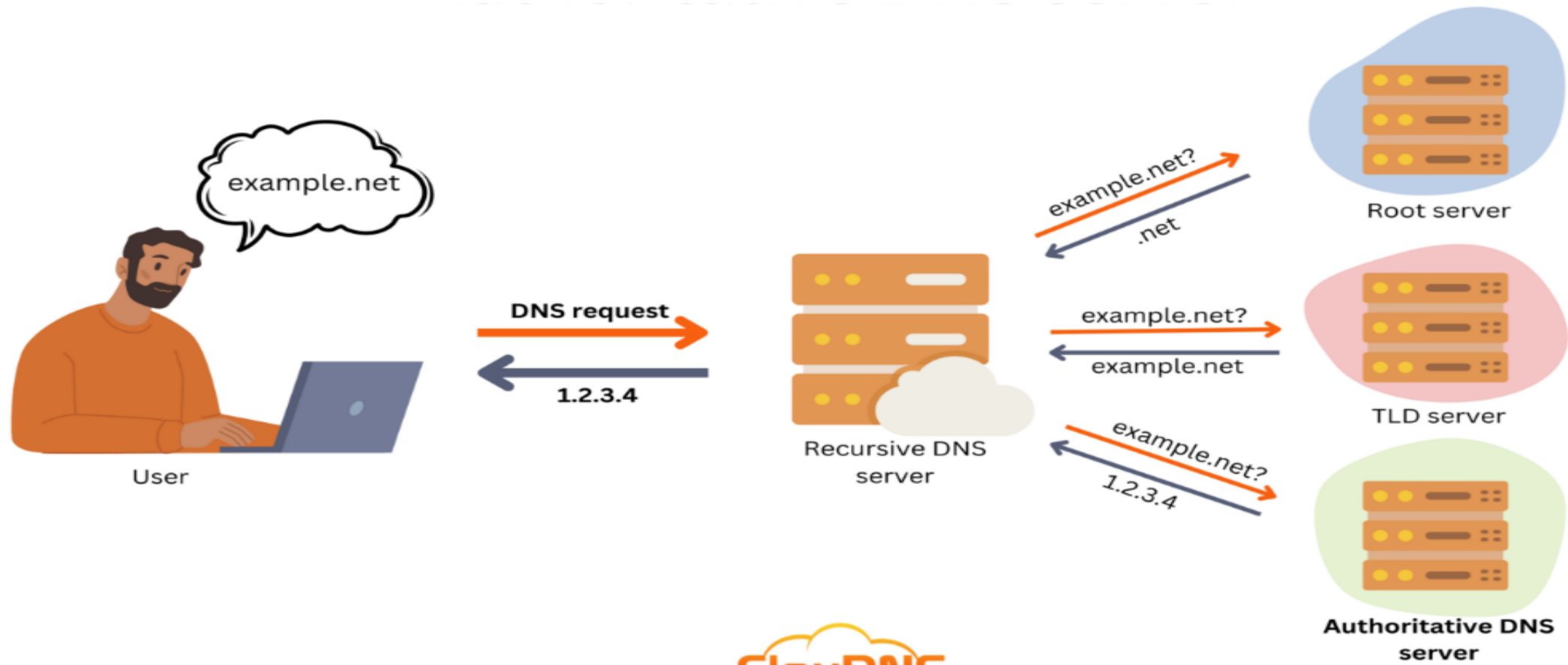
4.Subdomains

- A **subdomain** is a part of the domain name used to organize content or create separate sections of a website under a primary domain.

Examples:

- **blog.example.com:** A blog section under the main domain **example.com**.
- **store.example.com:** An e-commerce section under **example.com**.
- **us.example.com:** A subdomain tailored for users in the United States

Working Mechanism of DNS



- The user types a domain name (e.g., **www.example.com**) into the browser's address bar, initiating the DNS resolution process.
- The browser checks its **local cache** (stored from previous visits) to see if the domain's IP address is already available.
- If the IP address is not found in the browser cache, the system checks the **OS cache** for any previously stored DNS information.
- If the OS cache does not contain the IP address, the **router cache** is checked for any cached DNS information.
- If none of these caches provide the IP address, the system checks the **hosts file**, which may have manually configured mappings of domain names to IP addresses.
- If the IP address is not found in the hosts file, the query is sent to the **DNS Resolver** (usually provided by the ISP or a public DNS service like Google DNS or Cloudflare).
- The DNS Resolver acts as an intermediary and starts the DNS resolution process by querying other DNS servers.
- The **DNS Resolver** first contacts the **Root DNS Server**. The Root server doesn't have the exact IP address but knows which **Top-Level Domain (TLD) server** is responsible for the queried domain's extension (e.g., **.com**, **.org**).

- The Root server then directs the Resolver to the appropriate **TLD DNS Server** based on the domain's extension (e.g., for **example.com**, the query is directed to the **.com TLD server**).
- The **TLD DNS Server** doesn't have the full domain information but points the Resolver to the **Authoritative DNS Server** that holds the authoritative DNS records for the domain.
- The **Authoritative DNS Server** stores the DNS records (such as **A records**) for the domain, including the IP address associated with the domain name.
- The **Authoritative DNS Server** returns the **A record**, which contains the IP address for the domain (e.g., **192.0.2.1**).
- The DNS Resolver receives the IP address from the **Authoritative DNS Server** and sends it back to the user's computer.
- The user's computer now has the **IP address** of the domain and uses it to establish a connection with the real web server hosting the website.
- The browser sends an HTTP or HTTPS request to the web server using the obtained IP address.
- The web server processes the request, retrieves the requested resources (e.g., HTML, CSS, images), and sends the data back to the browser.
- The browser receives the content and displays the website on the user's screen, completing the DNS resolution process.

Domain Registration

- **Domain registration** is the process of acquiring a domain name for use on the internet.
- It is necessary for businesses, organizations, or individuals who wish to create a presence online, such as hosting a website or email service.
- The domain registration process is managed through domain registrars, which are accredited organizations that sell domain names and manage the associated DNS (Domain Name System) records.

Steps for Domain Registration

- **Choose a Domain Name:** Select a unique and relevant domain name that is easy to remember and aligns with your website or brand.
- **Select a Domain Registrar:** Choose a reliable, accredited domain registrar like GoDaddy, Namecheap, or others, where you can purchase the domain.
- **Check Domain Availability:** Use the registrar's search tool to verify if your desired domain name is available. If not, try alternatives.
- **Choose a Domain Extension:** Pick the appropriate **TLD** (e.g., **.com**, **.org**, **.net**) based on your website's target audience and purpose.
- **Register the Domain:** Provide your personal or business information and register the domain for 1 year or more.
- **Configure DNS:** Set up the necessary DNS records (such as A records or MX records) to link your domain with your website or email service.
- **Enable Domain Privacy (Optional):** Protect your personal information from public exposure by enabling **WHOIS Protection**.
- **Payment and Confirmation:** Complete the payment process to finalize the registration and receive confirmation from your registrar.

What is Web Hosting?

- **Web hosting** is a service that allows individuals or organizations to make their website accessible on the internet.
- Web hosting companies provide the infrastructure, resources, and technology needed to store website files and deliver them to users via the web.

Types of Web Hosting

Shared Hosting: Multiple websites are hosted on the same server and share its resources like CPU, RAM, and bandwidth.

Pros: Cost-effective, easy to set up.

Cons: Limited resources, performance can be affected by other websites on the server.

VPS (Virtual Private Server) Hosting: A virtual server is created on a physical server, giving you dedicated resources and more control than shared hosting.

Pros: More power, better control, and higher performance.

Cons: More expensive than shared hosting, requires technical expertise.

Dedicated Hosting: You get an entire physical server dedicated to your website.

Pros: Full control over the server, high performance, security.

Cons: Very expensive, requires advanced technical knowledge.

Cloud Hosting: Websites are hosted on a network of virtual servers (cloud) that are connected and can scale as needed.

Pros: Scalable, reliable, flexible, and cost-efficient for growing websites.

Cons: Can be more complex to manage.

Managed Hosting: The hosting provider manages the server, including maintenance, updates, and security.

Pros: No technical management required, secure and well-maintained.

Cons: More expensive than self-managed hosting.

WordPress Hosting: Specialized hosting for WordPress websites with optimized settings, automatic updates, and pre-configured tools.

Pros: Simplifies WordPress management, offers specialized tools.

Cons: Limited to WordPress websites.

Reseller Hosting: Allows you to resell hosting services to other customers while managing your own hosting plans.

Pros: Opportunity to start a hosting business.

Cons: Requires advanced management and support knowledge.

Choosing the Right Web Hosting

When selecting web hosting, consider factors like:

- **Website Size:** Small sites can go with shared hosting, while larger sites may need VPS or dedicated hosting.
- **Traffic Volume:** Sites with high traffic should look at cloud or dedicated hosting for better performance.
- **Technical Expertise:** If you're not technically inclined, managed hosting or shared hosting is the best option.
- **Budget:** Balance the hosting plan cost with the features and performance your website needs.

Web Hosting vs Domain Registration

Aspect	Web Hosting	Domain Registration
Function	Stores website files and data	Provides the address (URL) for your site
What You Buy	Server space and resources for your website	Rights to a specific domain name
Duration	Typically renewed annually or monthly	Typically registered for 1 year, renewable
Dependency	Needs a domain name to be accessed by users	Needs hosting to make the website live
Examples	GoDaddy Hosting, Bluehost, HostGator	Namecheap, GoDaddy, Google Domains

Web architecture

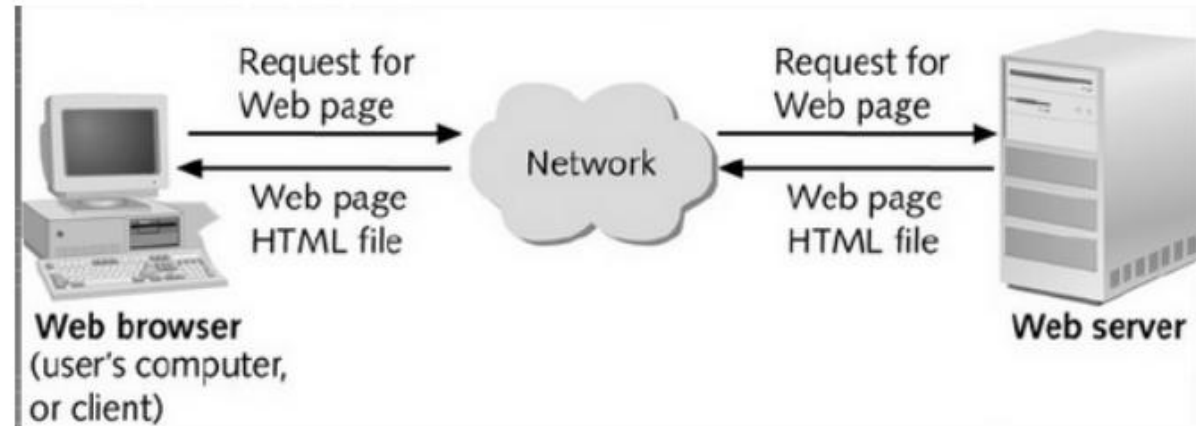
- **Web architecture** refers to the fundamental design and structure of the World Wide Web (WWW).
- It outlines the framework through which different systems, applications, and users communicate with each other.
- Web architecture provides the foundation for the interaction between servers, clients, and the infrastructure that supports the delivery and exchange of content over the internet.
- It involves the use of specific protocols, data formats, and addressing standards to ensure proper functioning and interoperability.

The key components :

1. **Data Transmission Protocols:** These protocols, such as **TCP/IP**, **HTTP**, and **HTTPS**, regulate how data is transmitted over the web. TCP/IP ensures end-to-end communication, while HTTP and HTTPS define the rules for transferring web content securely.
2. **Representation Formats:** Web content is structured using **HTML** (Hypertext Markup Language), **CSS** (Cascading Style Sheets), and **XML** (Extensible Markup Language). These formats specify how information is presented and exchanged between clients and servers.
3. **Addressing Standards: Uniform Resource Identifiers (URI) and Uniform Resource Locators (URL)** enable the identification and location of resources on the internet, ensuring users can access specific data.

Client-Server Model

- The **Client-Server Model** is one of the most fundamental and widely used architectures in computer networks, including the World Wide Web.
- It refers to a communication model where two entities, the **client** and the **server**, interact in a specific manner to share resources and services.
- The model is based on a **request-response** interaction pattern, where the client requests a service or resource, and the server responds to fulfill the request.
- **Client:** A device or application (like a web browser) that sends requests for services or resources.
- **Server:** A system that processes the client's requests and sends back the appropriate response or service.



Characteristics of the Client-Server Model

- **Separation of Roles:** Clients request services, servers provide them.
- **Request-Response Communication:** Clients send requests, and servers respond.
- **Centralized Resources:** Servers manage and store resources accessed by clients.
- **Scalability:** Servers can be scaled to handle more clients.
- **Modularity:** Clients and servers are distinct, simplifying management.
- **Security Management:** Centralized control over authentication and authorization.
- **Fault Isolation:** Client issues don't affect the server, but server failure impacts all clients.
- **Network Dependency:** Requires a stable network connection for communication.
- **Data Centralization:** Data is stored on the server for easy management and backup.

Examples:

- **Web Browsing:** The browser (client) requests a webpage from a web server.
- **Email:** An email client communicates with an email server to send or retrieve messages.
- **File Transfer:** An FTP client requests files from an FTP server.

Working of Client server model

Client Sends Request:

- The client (e.g., web browser, mobile app) initiates a request to the server. This could be a request for a webpage, file, or service.
- Example: A user types a URL into a browser, requesting a webpage.

Server Processes the Request:

- The server receives the client's request and processes it. This might involve querying a database, running a script, or retrieving a file from storage.
- Example: The web server checks for the requested webpage or fetches data from a database.

Server Sends Response:

- After processing, the server sends back a response to the client. This can include data (e.g., HTML for a webpage, or JSON for an API), an acknowledgment, or an error message.
- Example: The server sends the HTML, CSS, and JavaScript files needed to render the requested webpage.

Client Displays the Result:

- The client receives the response and processes it for display or action, like rendering the webpage or using the data in an application.
- Example: The browser renders the webpage for the user to view.

Advantages of the Client-Server Model

1. **Centralized Management:** Resources, security, and maintenance are controlled from a single server.
2. **Scalability:** Servers can be upgraded to handle more clients as demand grows.
3. **Resource Sharing:** Clients access shared resources on the server, reducing redundancy.
4. **Security:** Security policies (e.g., authentication) are managed centrally on the server.

Disadvantages of the Client-Server Model

1. **Single Point of Failure:** Server downtime affects all clients relying on it.
2. **Server Load:** Too many requests can overload the server, reducing performance.
3. **Cost of Server Maintenance:** Maintaining and upgrading servers can be costly.
4. **Network Dependency:** A stable network connection is required for communication; poor connectivity affects access.

Multi-Tier Architecture

- Tiered architecture, or multitier architecture, is a design approach used in software development to separate functionalities into distinct layers or tiers.
- Each tier has a specific role, allowing for better organization, scalability, and system maintainability

1. Presentation Tier:

- The presentation tier, also known as the user interface tier, is responsible for presenting information to users and accepting user inputs.
- Its main purpose is to handle user interactions and display data in a human-readable format. This tier provides the interface through which users interact with the application.

2. Application Tier:

- The application tier, also referred to as the business logic tier or middle tier, contains the core logic and functionality of the application.
- It processes user requests, implements business rules, performs computations, and coordinates the application's overall behavior.
- This tier acts as an intermediary between the presentation tier and the data tier.

3. Data Tier:

- The data tier, also known as the persistence tier or backend tier, is responsible for managing data storage, retrieval, and manipulation.
- It stores the application's data in a structured format, making it accessible to other tiers as needed.
- This tier ensures data integrity, security, and efficiency in data operations.

Advantages:

1. **Modularity:** Each tier can be developed and maintained independently.
2. **Scalability:** You can scale each tier independently based on the load. For example, if the database is under heavy use, it can be scaled separately.
3. **Maintainability:** Since the functionality is separated into distinct layers, maintenance and upgrades become easier.
4. **Flexibility:** Changes in one tier (e.g., upgrading the database) can be made without affecting other tiers.
5. **Security:** Sensitive data can be isolated in the data tier, with strict access control mechanisms.

Disadvantages:

1. **Complexity:** More tiers mean a more complex system. This can lead to additional overhead in terms of management and development.
2. **Performance Overhead:** Communication between multiple layers can introduce latency, especially in a networked system.
3. **Cost:** More resources (servers, hardware, etc.) may be needed to maintain the separate tiers, which could lead to increased operational costs.

1-Tier Architecture

- **1-Tier Architecture** is a software architecture where all the components of an application, including the user interface, business logic, and data storage, are integrated into a single system.
- There is no separation between the client and the server, and all the processing and data management tasks are handled on the same machine.

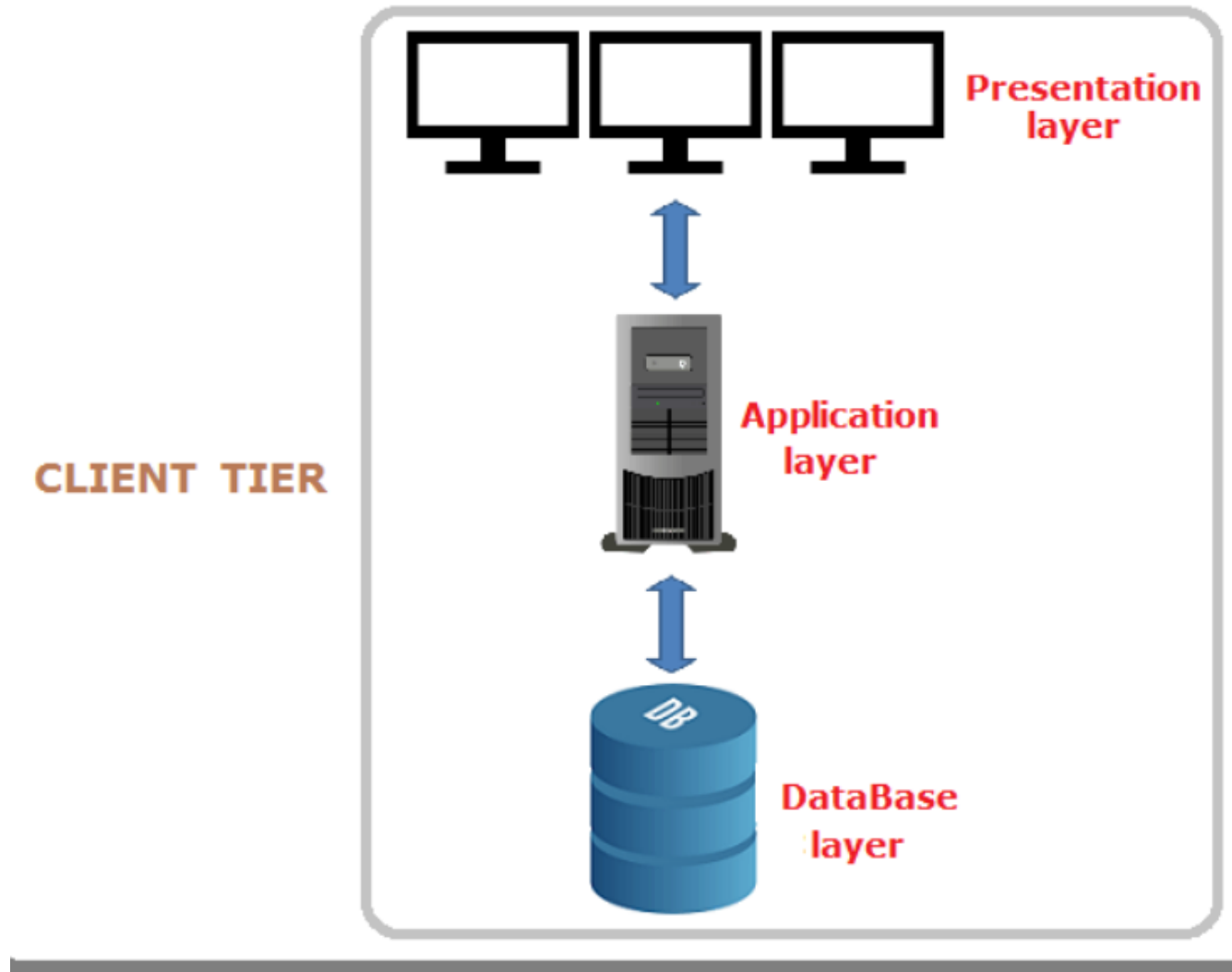
Example:

- A common example is Microsoft Excel. Everything from the user interface to the logic and data storage happens on the same device. The user enters data, performs calculations and saves files directly on their computer
- A **standalone database management system** (DBMS) that runs directly on a user's machine, without any server or client separation.

Features:

1. **Monolithic Design:** All layers (presentation, logic, and data) are combined into one system.
2. **Single System:** The application runs entirely on a single machine, including both the user interface and the database.
3. **No Client-Server Interaction:** Unlike multi-tier systems, there is no distinct client-server communication.
4. **Self-contained:** The system is entirely self-sufficient, requiring no external components or infrastructure.

ONE-TIER ARCHITECTURE



Advantages:

1. **Simplicity:** Easy to develop and deploy since all components are in one place.
2. **Cost-Effective:** Requires only one machine or server, reducing the need for complex infrastructure.
3. **Fast Development:** Ideal for small projects or prototypes, where quick development and deployment are essential.
4. **Low Maintenance:** Since everything is integrated, maintenance is relatively straightforward, especially in small-scale applications.

Limitations:

1. **Limited to Single User:** Only one person can use the application at a time.
2. **Poor Security:** Data and application are on the same system, posing security risks.
3. **No Centralized Control:** Data is stored locally, making management and backup difficult.
4. **Hard to Share Data:** Sharing between users is complex since everything is stored on one machine.

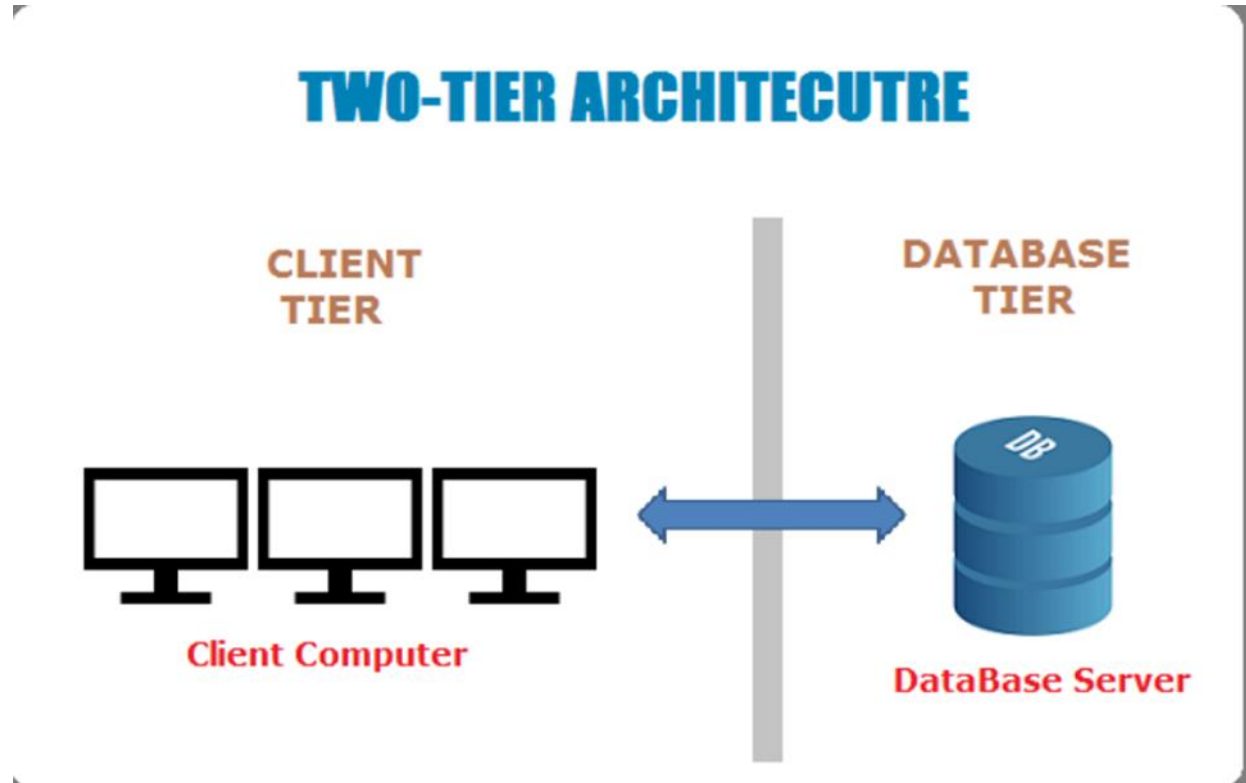
Two-Tier Architecture

- **Two-Tier Architecture** is a client-server architecture where the system is divided into two layers: the **client tier** (presentation and application logic) and the **server tier** (data management).
- The client directly communicates with the server to request and retrieve data, making it a straightforward and efficient architecture for simple applications.
- **Example:** A **desktop application** connecting to a **central server** to retrieve and update data, like an **inventory management system**.

Features:

- **Client-Server Model:** Client sends requests, server processes them and sends back data.
- **Direct Communication:** The client communicates directly with the server over a network.
- **Minimal Intermediate Layer:** No intermediary layers like business logic servers.
- **Decentralized Data Management:** The server handles data, while the client focuses on the UI.

Working Mechanism :



1. Client Layer (User Interface & Application Logic):

- The client is responsible for **user interaction** and **presentation**. It provides the user interface (UI) where users enter data, view information, and interact with the system.
- The client also handles **basic application logic**, such as validating inputs or managing session data.
- When the user takes an action (e.g., logging in or submitting a query), the client **sends a request** to the server for processing.
- **Example:** The user enters login credentials in a banking app (client).

2. Server Layer (Data Management & Processing):

- The server handles the **business logic** and **data storage**.
- The server **receives requests** from the client, processes them (e.g., validating credentials, querying a database), and interacts with the **database** to fetch or update data.
- The server then **sends a response** back to the client, which may include data (e.g., user account details) or a status message (e.g., "Login successful").
- **Example:** The server verifies the login credentials and retrieves the user's balance from the database.

3. Client Displays Data:

- The client **receives the response** from the server (such as the requested data or a confirmation message).
- It then updates the **user interface** to reflect the response, showing the requested information or providing feedback (e.g., successful login, account details).
- **Example:** The banking app displays the account balance after receiving it from the server.

Advantages :

1. **Simplicity:** Easy to design and implement with only two layers (client and server).
2. **Cost-Effective:** Fewer components reduce infrastructure costs.
3. **Faster Data Retrieval:** Direct client-server communication leads to quicker data exchange.
4. **Quick Deployment:** Ideal for small applications or prototypes.
5. **Easy Maintenance:** Fewer layers make it easier to manage and troubleshoot.

Limitations :

1. **Limited Scalability:** Server can become overloaded with more clients.
2. **Security Risks:** Direct client-server communication exposes the system to vulnerabilities.
3. **Tightly Coupled Layers:** Changes in one layer may require modifications in the other.
4. **No Centralized Business Logic:** Logic in the client can be hard to manage and secure.
5. **No Load Balancing:** High traffic can overwhelm the server without load distribution.

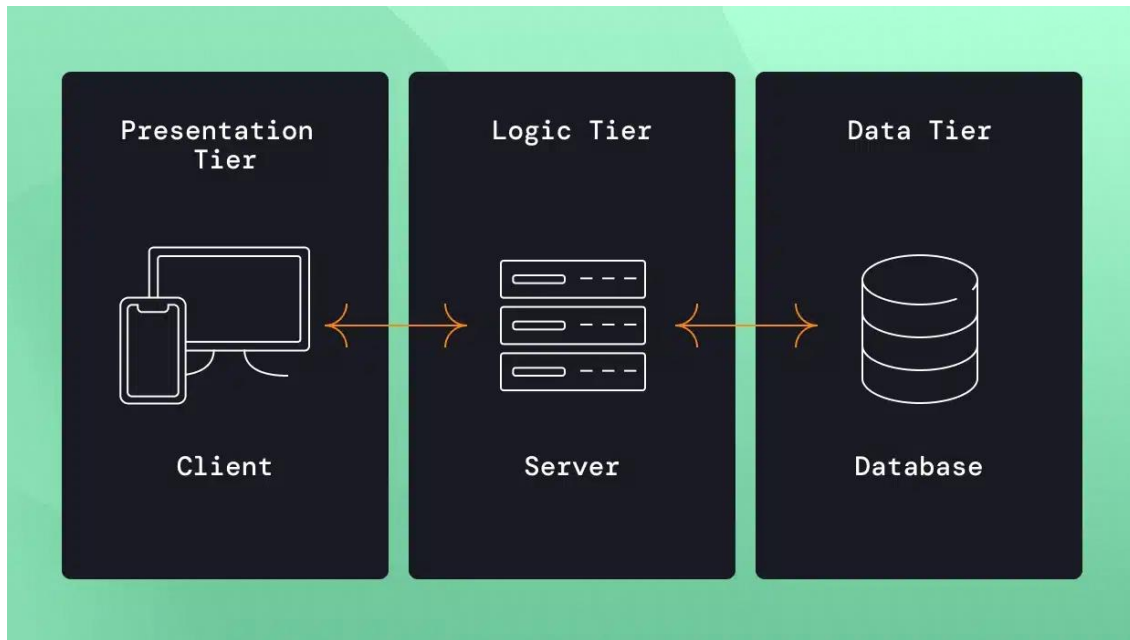
Three-Tier Architecture

- **Three-Tier Architecture** is a software architectural model that divides an application into three distinct layers:
 1. **Presentation Tier** (Client Layer): Manages user interactions and the user interface.
 2. **Logic Tier** (Application Layer): Handles the application's business logic and processes requests.
 3. **Data Tier** (Database Layer): Responsible for data storage and retrieval.
- This separation of concerns allows for improved scalability, security, and maintainability by isolating each layer's responsibilities.

Features :

- **Separation of Concerns:** Each tier has a specific responsibility, ensuring clear separation of functionality.
- **Modularity:** Changes to one layer (e.g., updating business logic) can be made independently without affecting others.
- **Scalability:** Each tier can be scaled independently to handle increasing workloads.
- **Maintainability:** Since each tier is isolated, the system is easier to maintain and update.
- **Security:** Sensitive data and business logic are separated from the user interface, making the system more secure.

Working Mechanism of Three-Tier Architecture



Flow of Three-Tier Architecture

1.

1. Presentation Tier (User Interface)



User Request

Send Input to Logic Tier

Display Results

Show Response to User

2.

2. Logic Tier (Business Logic)



Data Query

Request Data from Database



Process Request

Apply Business Rules

3.

3. Data Tier (Database)



Send Data

Return Data to Logic Tier

Fetch/Update Data

Access or Modify Database

1. Presentation Tier (Client Layer)

- This is the **user interface (UI)** layer where users interact with the system. It can be a web browser, mobile app, or desktop application.
- It collects input from users (e.g., login credentials, search terms) and sends these requests to the **Logic Tier**.
- It displays the results received from the **Logic Tier**, such as user information, search results, or error messages.
- **Example:** A user opens an online shopping website and enters a search query for products.

2. Logic Tier (Application Layer)

- The **middle layer** that processes the business logic. It acts as an intermediary between the **presentation tier** and the **data tier**.
- It receives requests from the **Presentation Tier** and processes them according to the business logic.
- It may include operations like validating data, running calculations, or making decisions based on user input.
- It communicates with the **Data Tier** to fetch, modify, or store data based on the processed request.
- After processing, it sends the appropriate data or status back to the **Presentation Tier**.
- **Example:** When the user submits a search query, the logic tier processes it (e.g., filtering, applying business rules) and queries the **Data Tier** for relevant products.

3. Data Tier (Database Layer)

- This is the **data storage** layer, typically represented by a **database** where all application data is stored.
- It handles **data storage, retrieval, and management**.
- The **Logic Tier** communicates with this layer to get the required data (e.g., retrieving product details, user information) or store new data (e.g., saving order details).
- It can use a **Database Management System (DBMS)** like MySQL, SQL Server, or Oracle to manage large volumes of data.
- **Example:** When the logic tier sends a query for product details, the **Data Tier** retrieves the relevant data from the database and sends it back.

Advantages of 3-Tier Architecture

1. **Separation of Concerns:** Clear division between UI, business logic, and data management, improving organization and maintainability.
2. **Scalability:** Each tier can be independently scaled to meet growing demand (e.g., adding more servers).
3. **Flexibility:** Changes in one tier (e.g., business logic) don't affect the others, allowing for easy adaptation to new requirements.
4. **Security:** Sensitive data is isolated in the data tier, reducing exposure to security risks.
5. **Maintainability:** Independent tiers allow for easier updates or fixes without impacting the entire system.

Limitations of 3-Tier Architecture

1. **Complexity:** More complex to design and implement compared to simpler architectures like Two-Tier.
2. **Performance Overhead:** Additional layers introduce potential bottlenecks in communication between tiers.
3. **Cost:** Requires more infrastructure and resources, making it more expensive to set up and maintain.
4. **Latency:** Data must pass through multiple layers, which can increase latency, especially in high-traffic systems.

Best Practices for Good Web Application Architecture

- **Separation of Concerns (SoC):** Divide the application into distinct layers (UI, business logic, and data management) for easier maintenance and better organization.
- **Scalability:** Design the system to scale by adding more servers or upgrading infrastructure as user demand increases.
- **Caching:** Use caching (e.g., Redis, Memcached) to reduce server load and speed up data retrieval.
- **Security:** Implement encryption, input validation, authentication, and authorization mechanisms to protect data and user privacy.
- **Responsive Design:** Ensure the application is accessible on all devices (desktop, mobile, tablet) by using responsive design principles.
- **Performance Optimization:** Minimize load times by optimizing images, compressing assets, and using techniques like lazy loading.
- **Version Control & CI/CD:** Use version control systems (e.g., Git) and automate deployment processes with CI/CD for smooth updates and continuous integration.

Contd...

- **Microservices:** Break large applications into smaller, independent services to improve scalability and flexibility.
- **Database Optimization:** Use appropriate database models (SQL vs NoSQL) and optimize queries with indexing and caching.
- **Load Balancing:** Distribute traffic across multiple servers to prevent overload on any single server and ensure high availability.
- **Logging and Monitoring:** Implement logging and monitoring tools to track application health and troubleshoot performance issues.
- **Modular Codebase:** Write modular, clean code to ensure easy updates, maintenance, and future enhancements.
- **Regular Backups:** Perform regular backups of critical data to avoid data loss from system failures.
- **Documentation:** Maintain up-to-date documentation for the codebase and API to support future development and collaboration.

Presentation:

Architecting Web Applications

Characteristics

Needs

Pros and Cons